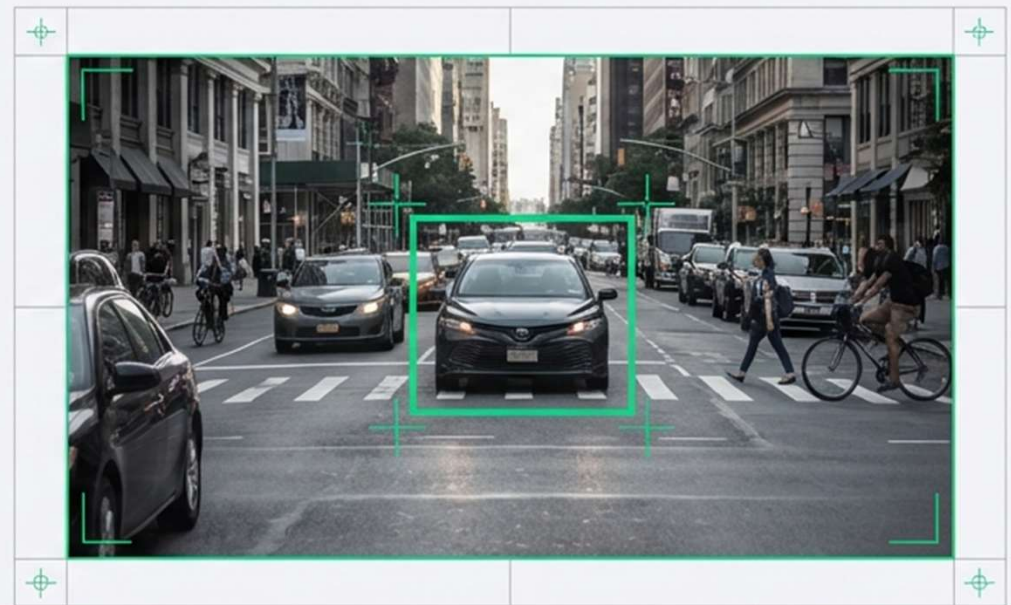


You Only Look Once

Revolutionizing Object Detection Architecture

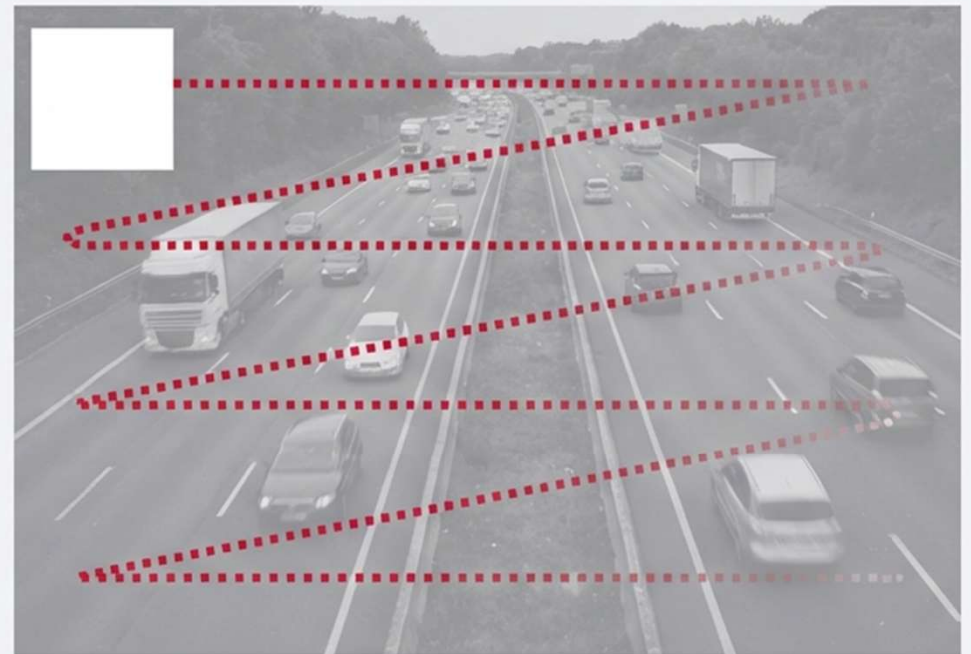


Dr. Jawwad Ahmed Shamsi
FAST-NUCES

The Sliding Window Bottleneck

Traditional object detection relied on scanning an image sequentially, creating three critical points of failure:

1. Multiple objects caught within a single scanning window.
2. Large objects spanning across multiple separate windows.
3. Unbalanced bounding rectangles that fail to capture real-world object shapes.



The Grid: A Single-Shot Solution

Instead of scanning, YOLO divides the entire image into an $S \times S$ grid simultaneously.

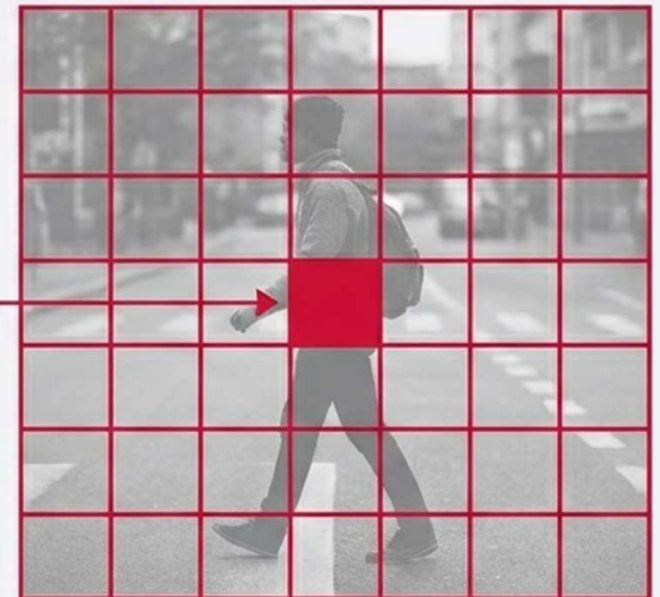
The Golden Rule

The single grid cell that contains the center of an object is exclusively responsible for detecting it.

Slow Sequential Scan



Simultaneous Grid Overlay

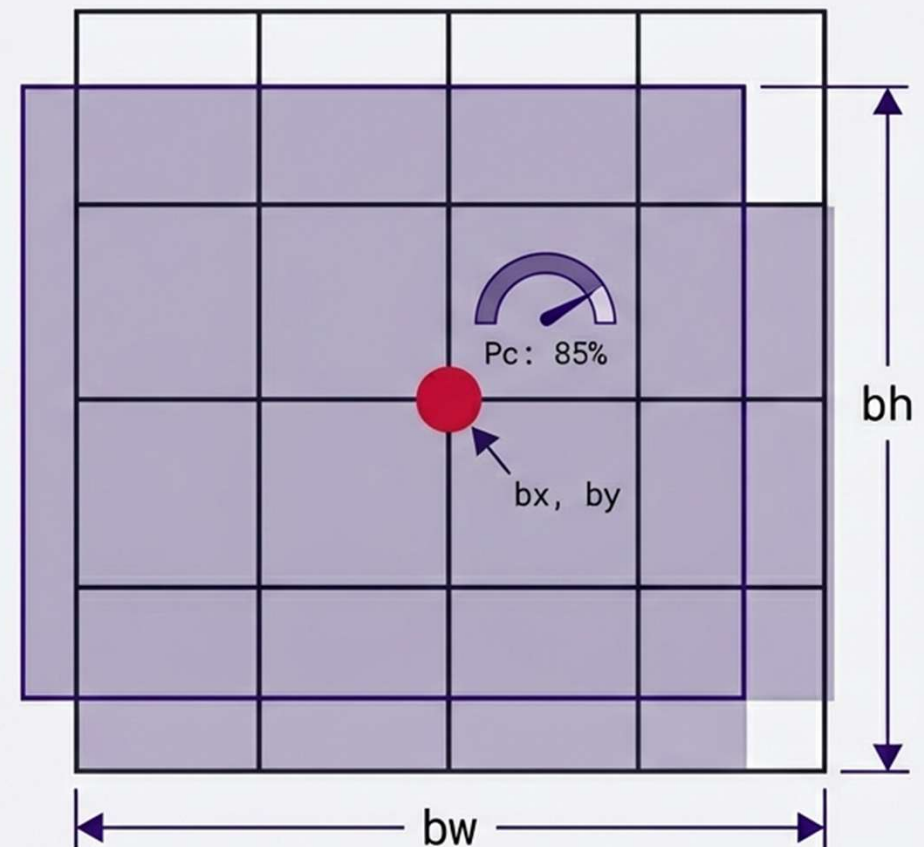


Dr. Jawwad Ahmed Shamsi
FAST-NUCES

Anatomy of a Prediction

Each responsible cell outputs a specific set of variables to draw the bounding box:

- P_c : Probability that an object actually exists in the box.
- b_x , b_y : Center coordinates of the object. Strictly bounded between 0 and 1 relative to the cell.
- b_w , b_h : Width and height of the bounding box. Relative to the entire image, allowing it to span multiple grids.



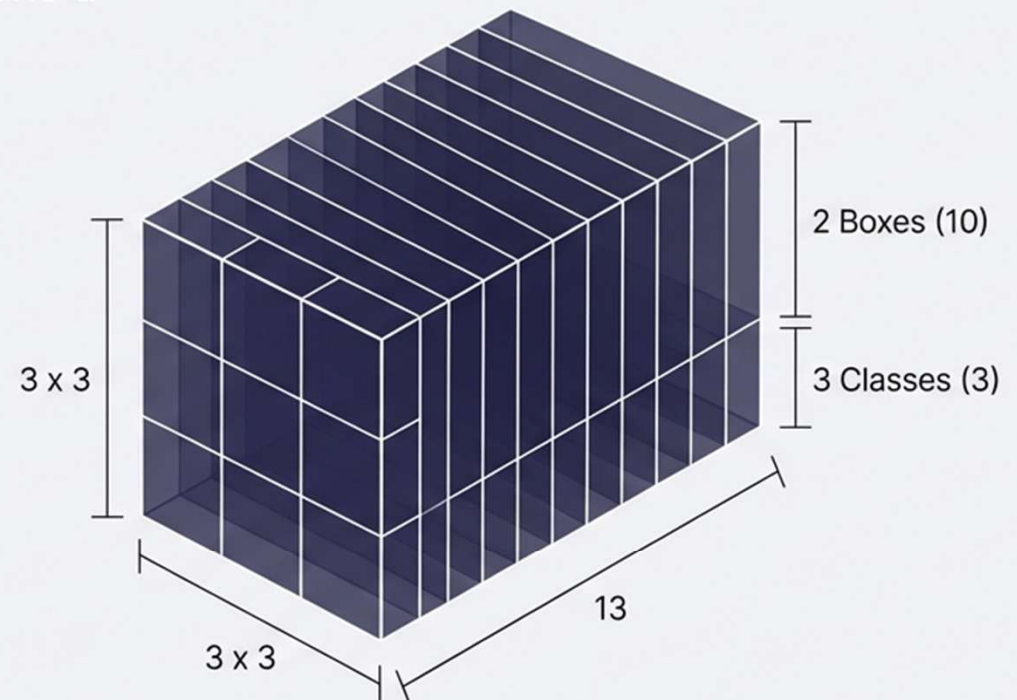
The Output Tensor Dimension

The grid and bounding box predictions fuse into a single, unified output representation:

Formula: $S \times S \times (B * 5 + C)$

- $S \times S$: The Grid Size
- B : Number of boxes predicted per cell
- 5: Core variables (P_c , b_x , b_y , b_w , b_h)
- C : Number of classes

Example: A 3×3 grid, predicting 2 boxes per cell, across 3 target classes, results in an output tensor size of $3 \times 3 \times 13$.



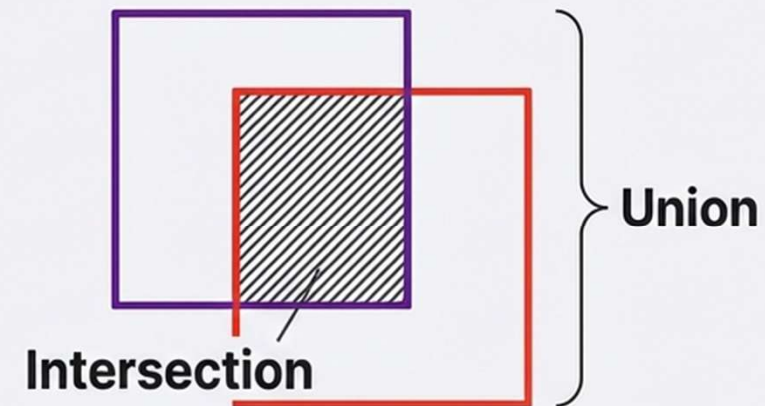
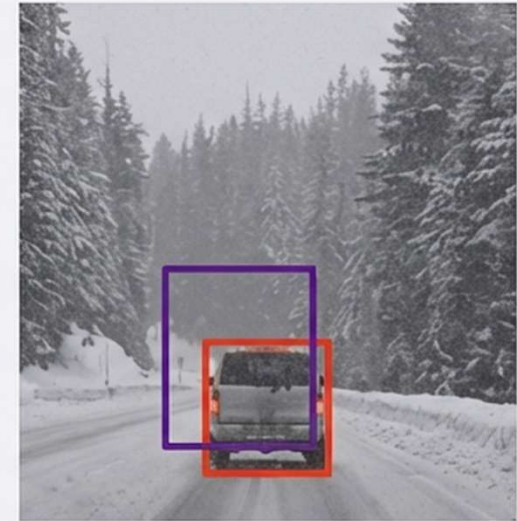
Measuring Accuracy: Intersection Over Union

IoU measures the exact overlap between the algorithm's predicted bounding box and the actual ground truth bounding box.

$$\text{IoU} = |\text{Bpred} \cap \text{Bgt}| / |\text{Bpred} \cup \text{Bgt}|$$

Target: Correct predictions require an $\text{IoU} \geq 0.5$

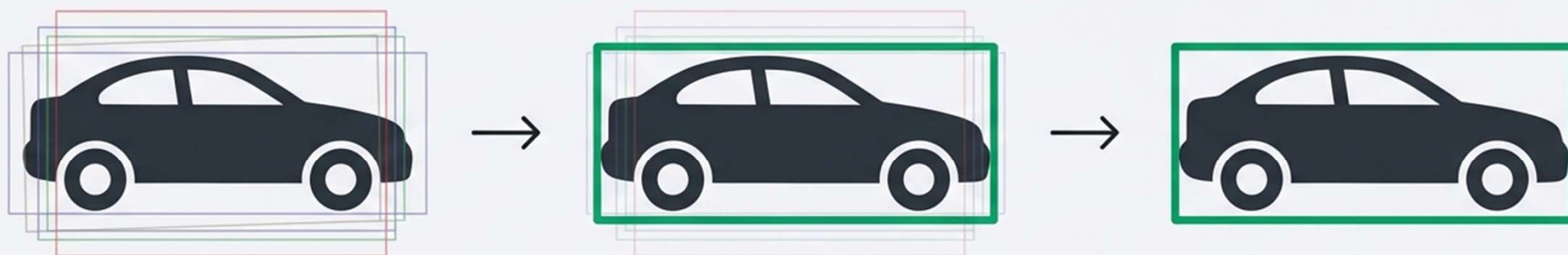
$$\text{Final Score} = P(\text{class} \mid \text{object}) * P(\text{object}) * \text{IoU}$$



The Duplicate Problem: Non-max Suppression

When multiple cells detect the same object, the NMS algorithm acts as the ultimate filter:

1. Discard boxes below baseline confidence threshold.
2. Select the box with the absolute highest confidence score.
3. Remove all other competing boxes with high IoU overlap.
4. Repeat until no overlapping duplicate boxes remain.



Multiple Detections

Identify Highest Confidence

Suppress Overlaps

Evaluating Success: Mean Average Precision

To benchmark YOLO's performance, we rely on Mean Average Precision (mAP), built on two foundational metrics:

Precision

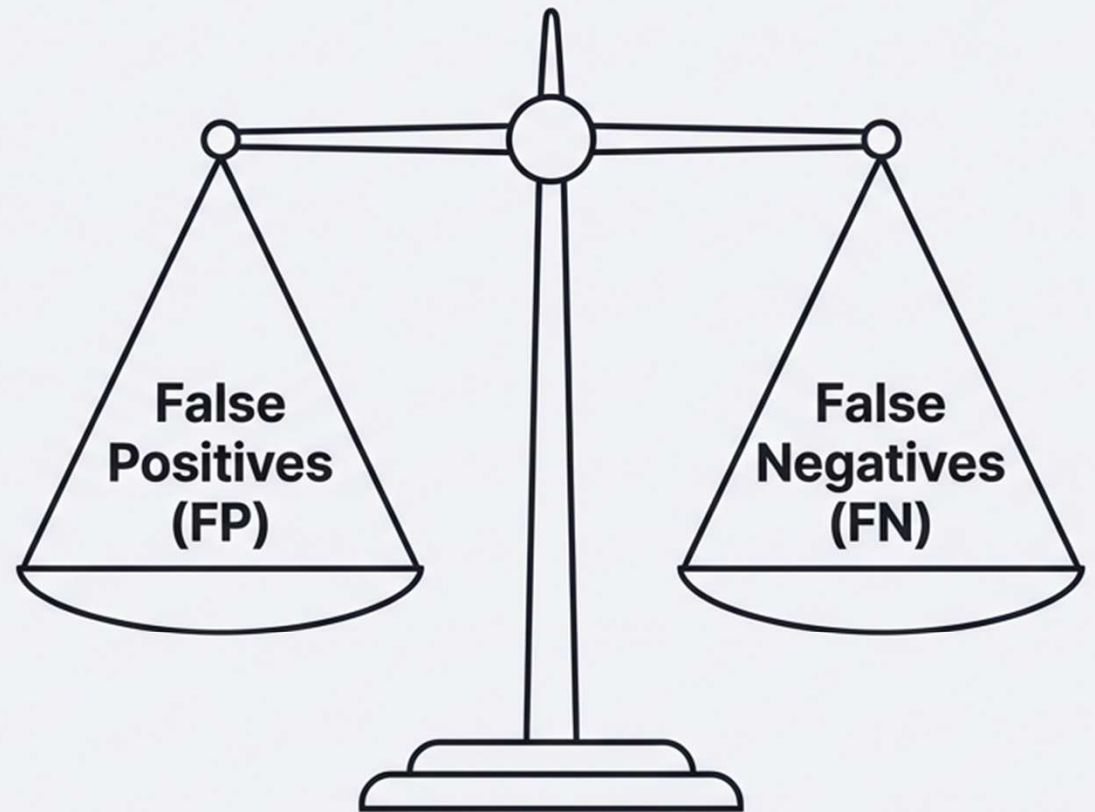
$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Out of all the objects YOLO claimed were cars, how many were actually cars?

Recall

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Out of all the actual cars in the image, how many did YOLO successfully find?

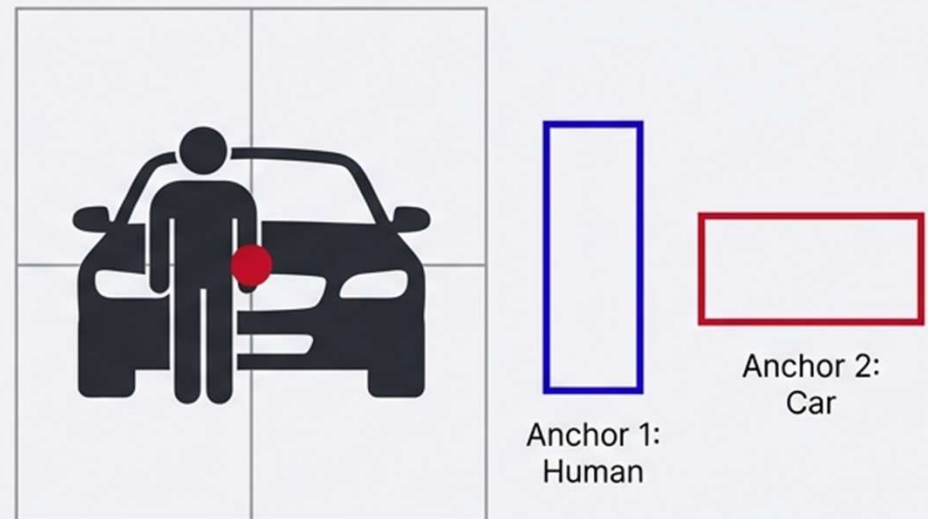


The Overlap Challenge: Enter Anchor Boxes

The Limitation: Standard YOLO allows each grid cell to predict only one object. If a pedestrian stands directly in front of a car, they share a center point, causing detection failure.

The Solution: Anchor Boxes. These are pre-assumed, common shapes provided to the network.

Anchor boxes can be hand-selected or derived using K-means clustering on the dataset to find the most frequent object proportions.



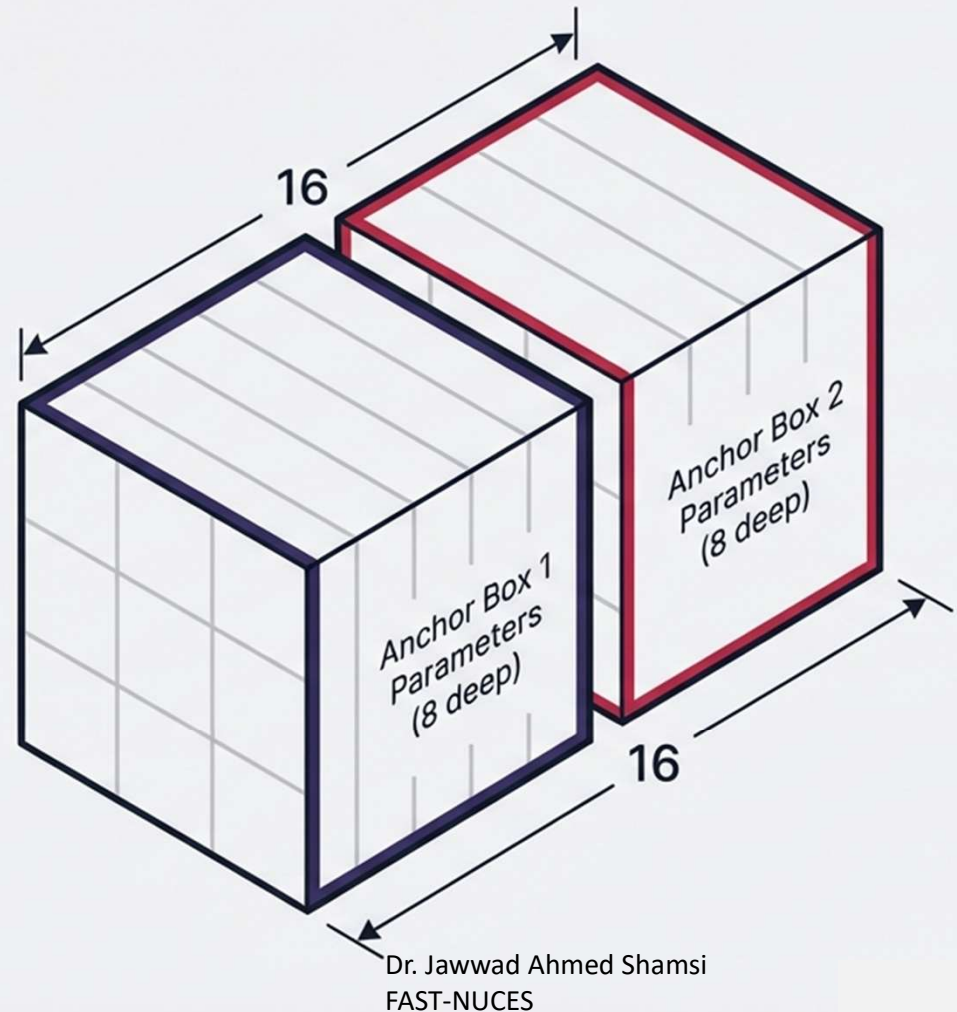
Expanding the Tensor for Multiple Predictions

By integrating anchor boxes, objects are assigned to a pair: (Grid Cell, Anchor Box). This modifies our initial output equation to accommodate multiple object predictions per cell.

New Tensor Example: Using a 3x3 grid, with 2 distinct anchor boxes, evaluating 3 classes:

$$3 \times 3 \times 2 \times 8$$

Limitation: A cell cannot detect three objects if it only has two anchor boxes, nor can it detect two objects of the exact same anchor shape.



Bounding Box Parameterization

The network doesn't use the raw anchor box; it predicts offsets (tx, ty, tw, th) to refine the final size:

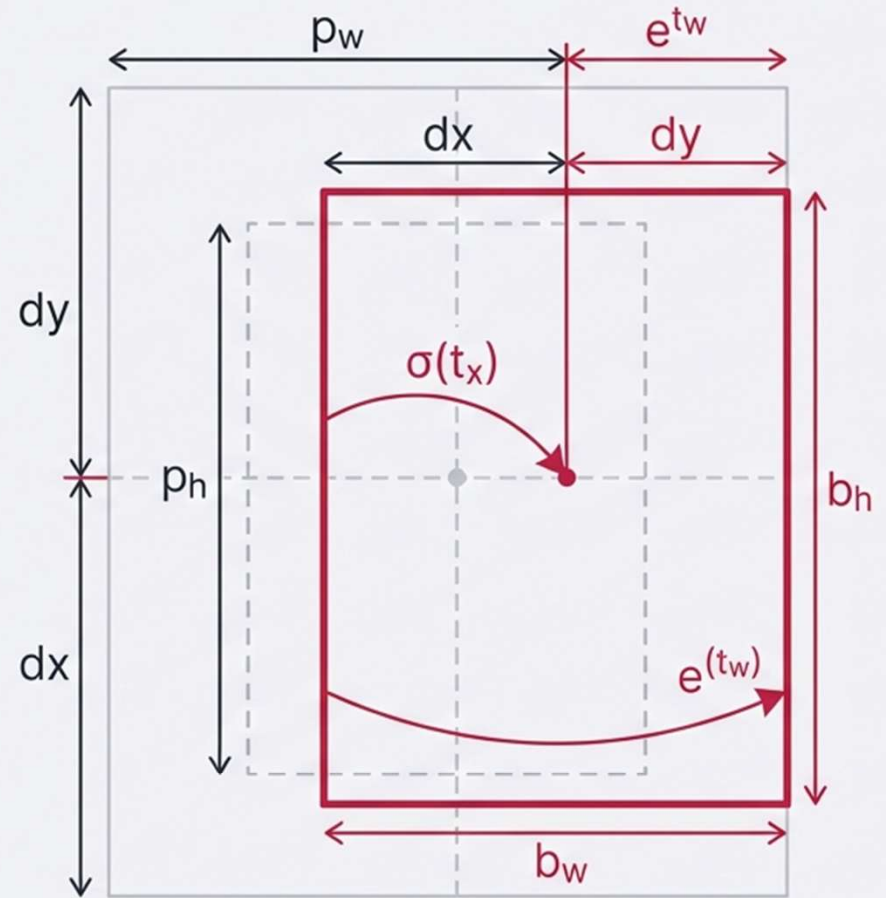
$$b_X = \sigma(t_X) + c_X$$

$$b_y = \sigma(t_y) + c_y$$

$$bw = pw * e^{(tw)}$$

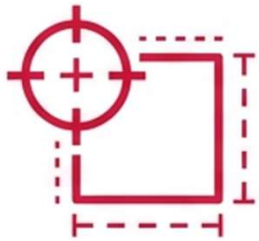
$$bh = ph * e^{(th)}$$

(cx, cy: grid cell top-left corner | pw, ph: anchor box dimensions | σ : sigmoid function)



The YOLO Loss Function

The network is trained using a loss function L composed of three distinct pillars, balanced by weighting factors:



1. Localization Loss

How accurately did we draw the box? Penalizes errors in coordinates (x, y) and dimensions (w, h) .



2. Confidence Loss

Is there actually an object there? Penalizes false positives and false negatives.



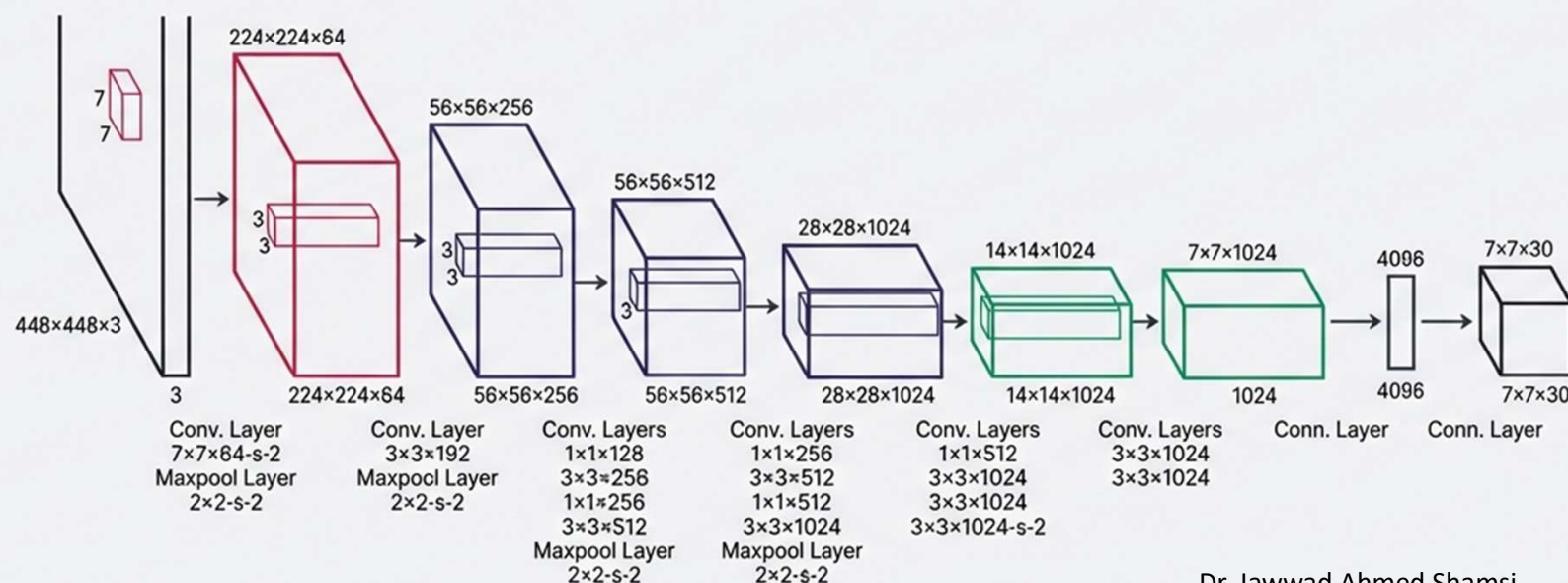
3. Classification Loss

Did we guess the right object? Penalizes incorrect class predictions.

The Detection Engine: YOLO Architecture

The underlying neural network executing the YOLO algorithms relies heavily on transfer learning:

- **The Structure:** 24 Convolutional layers followed by 2 Fully Connected layers.
- **Efficiency:** Alternating 1×1 convolutional layers reduce the feature space from preceding layers.
- **Transfer Learning:** Pre-trained on the ImageNet task at 224×224 , then doubled to 448×448 for detection.



Dr. Jawwad Ahmed Shamsi
FAST-NUCES

The Complete Pipeline

By framing object detection as a single regression problem, YOLO eliminates the need for complex, multi-stage pipelines.

From the moment an image hits the $S \times S$ grid, through the 24-layer convolutional architecture, to the application of Anchor Boxes and Non-Max Suppression, the entire image is evaluated in a single forward pass.

The result is a system that achieves profound accuracy without sacrificing real-time processing speed.

